

Tworzenie i usuwanie instancji klasy.

ZADANIE

Napisać klasę o nazwie Bug taką, żeby zawierała licznik wskazujący aktualną liczbę powołanych do życia obiektów, identyfikator (lokalną zmienną obiektu, do której przypiszemy aktualny powiększony licznik). Licznik powinien rosnąć wraz z wywołaniem `__init__` oraz maleć z wywołaniem `__del__` (uwaga: współdzielony licznik będziemy używać w zapisie `Bug.licznik`). Proszę też zdefiniować `__str__` wypisującą licznik i bieżące id. Proszę też napisać jakiś opisowy komentarz w klasie (w formie docstring). Finalnie, niech dla kodu:

```
bugs = []
for i in range(100):
    bugs.append(Bug())
    print(bugs[-1])
```

wypisują się licznik, identyfikator, a przy niszczeniu obiektu w `__del__` niech będzie też print informacji typu 'Koniec', licznik, identyfikator.

```
class Bug:
    licznik = 0 # Wspólny licznik dla wszystkich obiektów klasy Bug

    def __init__(self):
        Bug.licznik += 1 # Zwiększamy licznik przy tworzeniu obiektu
        self.id = Bug.licznik # Przypisujemy unikalny identyfikator

    def __del__(self):
        print(f"Koniec, licznik: {Bug.licznik}, id: {self.id}") # Informacja przy usuwaniu
        Bug.licznik -= 1 # Zmniejszamy licznik przy usuwaniu obiektu

    def __str__(self):
        return f"Licznik: {Bug.licznik}, id: {self.id}" # Reprezentacja obiektu
```

 ZADANIE

Śledzenie obiektów.

```
>>> import sys
>>> a = 1
>>> sys.getrefcount(a)
4294967295
>>> # -5 do 256
>>> a = 257
>>> sys.getrefcount(a)
2
>>> b = a
>>> sys.getrefcount(a)
3
>>> del a
>>> sys.getrefcount(a)
Traceback (most recent call last):
  File "<python-input-117>", line 1, in <module>
    sys.getrefcount(a)
                    ^
NameError: name 'a' is not defined
>>> sys.getrefcount(b)
2
```

Wywołanie `sys.getrefcount(x)` zawsze zwraca **o 1 więcej**, bo na czas wywołania funkcji `x` jest jeszcze **dodatkową referencją (argument funkcji)**.

`weakref.ref(a)` daje **uchwyt-observatora**, który:

- zwraca obiekt, **jeśli ten jeszcze żyje**
- zwraca `None`, kiedy obiekt został już zniszczony

```
import weakref
```

```
class Foo:
    pass
```

```
a = Foo()
```

```
w = weakref.ref(a)
```

```
print(w() is a) # True – obiekt żyje
```

```
del a
```

```
print(w())      # None – obiekt już skasowany
```

w **nie jest** obiektem `b`, tylko **funkcją/uchwytem**, który zwraca `a` – dopóki `a` żyje
 Żeby z tego **weakrefa** dostać z powrotem oryginalny obiekt, trzeba go **wywołać jak funkcję**: `w()`

weakref nie działa z:

`int`, `float`, `bool`

`str`, `bytes`, `tuple` (zwykłe, niemodyfikowalne)

generalnie większością **prostych, niemutowalnych typów wbudowanych**

Operatory klasy

 ZADANIE

```
import math
```

```
class Zespolona:
```

```
    def __init__(self, r=0.0, i=0.0):
        """Inicjalizacja liczby zespolonej: r + i*j."""
        self.r = float(r)
        self.i = float(i)
```

```
    def conjugate(self):
        """Zwraca sprzężenie zespolone liczby."""
        return Zespolona(self.r, -self.i)
```

```
    def argz(self):
        """Zwraca argument (fazę) liczby zespolonej w radianach."""
        return math.atan2(self.i, self.r)
```

```
    def __repr__(self):
        """Reprezentacja „techniczna”, przydatna w debugowaniu."""
        return f"Zespolona({self.r}, {self.i})"
```

```
    def __str__(self):
        """Ładna reprezentacja tekstowa, np. '1.0 + 2.0i'."""
        znak = "+" if self.i >= 0 else "-"
        return f"{self.r} {znak} {abs(self.i)}i"
```

```
# Przykład gotowej metody:
```

```
def __add__(self, other):
    """Dodawanie dwóch liczb zespolonych: self + other.
```

```
    """
    Jeśli other nie jest obiektem klasy Zespolona,
    metoda powinna zwrócić NotImplemented.
```

```
    """
    if not isinstance(other, Zespolona):
        return NotImplemented
    return Zespolona(self.r + other.r, self.i + other.i)
```

```
if __name__ == "__main__":
```

```
    # Proste testy (do uruchomienia po implementacji metod):
```

```
    a = Zespolona(1, 2)
    b = Zespolona(3, -1)
```

```
    print("a =", a)
```

```
    print("b =", b)
```

```
    print("a + b =", a + b)
```

```
    print("a - b =", a - b)      # zadziała po implementacji __sub__
```

```
    print("a == b ?", a == b)    # zadziała po implementacji __eq__
```

```
    print("a == Zespolona(1, 2) ?", a == Zespolona(1, 2))
```

```
# Zadanie: zaimplementuj poniższe metody na wzór __add__.
```

```
def __sub__(self, other):
```

```
    """Odejmowanie: self - other."""
```

```
    pass
```

```
def __eq__(self, other):
```

```
    """Porównanie == dwóch liczb zespolonych."""
```

```
    pass
```

indeksy

 ZADANIE

```
class TicTacToeBoard:
    def __init__(self):
        # 3x3 board filled with dots
        self._board = [["." for _ in range(3)] for _ in range(3)]

    def __getitem__(self, pos):
        """Read a field: board[row, col]."""
        # pos is a tuple (row, col)
        pass

    def __setitem__(self, pos, value):
        """Write a field: board[row, col] = value."""
        # pos is a tuple (row, col)
        pass

    def __str__(self):
        """String representation of the board."""
        return "\n".join(" ".join(row) for row in self._board)
```

```
if __name__ == "__main__":
    board = TicTacToeBoard()
```

```
    board = TicTacToeBoard()
    board[0, 0] = "X"
    board[1, 1] = "O"
    board[2, 2] = "X"
```

Zadanie: zaimplementuj poniższe metody

```
def __getitem__(self, pos):
    """Read a field: board[row, col]."""
    # pos is a tuple (row, col)
    pass

def __setitem__(self, pos, value):
    """Write a field: board[row, col] = value."""
    # pos is a tuple (row, col)
    pass
```